

# **FNCE 5352 — Lecture 5 (Part 2): Errors, Thresholds, ROC**

Evaluating classifiers (credit risk lens)

## ## Classification Metrics {.smaller}

### Definitions:

- **Hard Predictions:** provide the final classification result directly, indicating the class to which the input data is most likely to belong, without showing any uncertainty or probability of the decision.
  - Example: A model predicts an obligor is either “defaulter” (1) or “not a defualter” (0), with no indication of how likely it is to be a defaulter.
- **Soft Predictions:** provide a probability distribution over all possible classes, indicating the likelihood that the input data belongs to each class.
  - Example: A model predicts an obligor has a 90% probability of being a “defaulter” and a 10% probability of being “not a defaulter”.

# Confusion Matrix

For Hard Predictions, we can create a confusion matrix

|                  |              | Actual Values |              |
|------------------|--------------|---------------|--------------|
|                  |              | Positive (1)  | Negative (0) |
| Predicted Values | Positive (1) | TP            | FP           |
|                  | Negative (0) | FN            | TN           |

# Components of Confusion Matrix

1. **True Positives (TP):** Instances correctly predicted as positive.
2. **True Negatives (TN):** Instances correctly predicted as negative.
3. **False Positives (FP):** Instances incorrectly predicted as positive, also known as Type I error.
4. **False Negatives (FN):** Instances incorrectly predicted as negative, also known as Type II error.

# Derived Metrics from Confusion Matrix

From the confusion matrix, several important metrics can be calculated, including:

- **Accuracy:** Overall correctness of the model.  $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$
- **Precision:** Proportion of positive identifications that were actually correct.  $Precision = \frac{TP}{TP+FP}$
- **Recall (Sensitivity or True Positive Rate):** Proportion of actual positives that were identified correctly.  
 $Recall = \frac{TP}{TP+FN}$
- **Specificity (True Negative Rate):** Proportion of actual negatives that were identified correctly.  
 $Specificity = \frac{TN}{TN+FP}$
- **F1 Score:** Harmonic mean of precision and recall, providing a single metric to assess the balance between them.  
 $F1 = 2 \times \frac{Precision * Recall}{Precision + Recall}$

# Mapping Soft Predictions into Hard Predictions

For two classes, the customary cutoff to map probabilities into predictions is 50%. If the probability of class 1 is  $\geq 50\%$ , they would be labelled as *Class1*

What happens if you change the cutoff?

- Increasing makes it harder to be called *Class1*  $\Rightarrow$  fewer predicted events, specificity  $\uparrow$ , sensitivity  $\downarrow$
- Decreasing makes it easier to be called *Class1*  $\Rightarrow$  more predicted events, specificity  $\downarrow$ , sensitivity  $\uparrow$

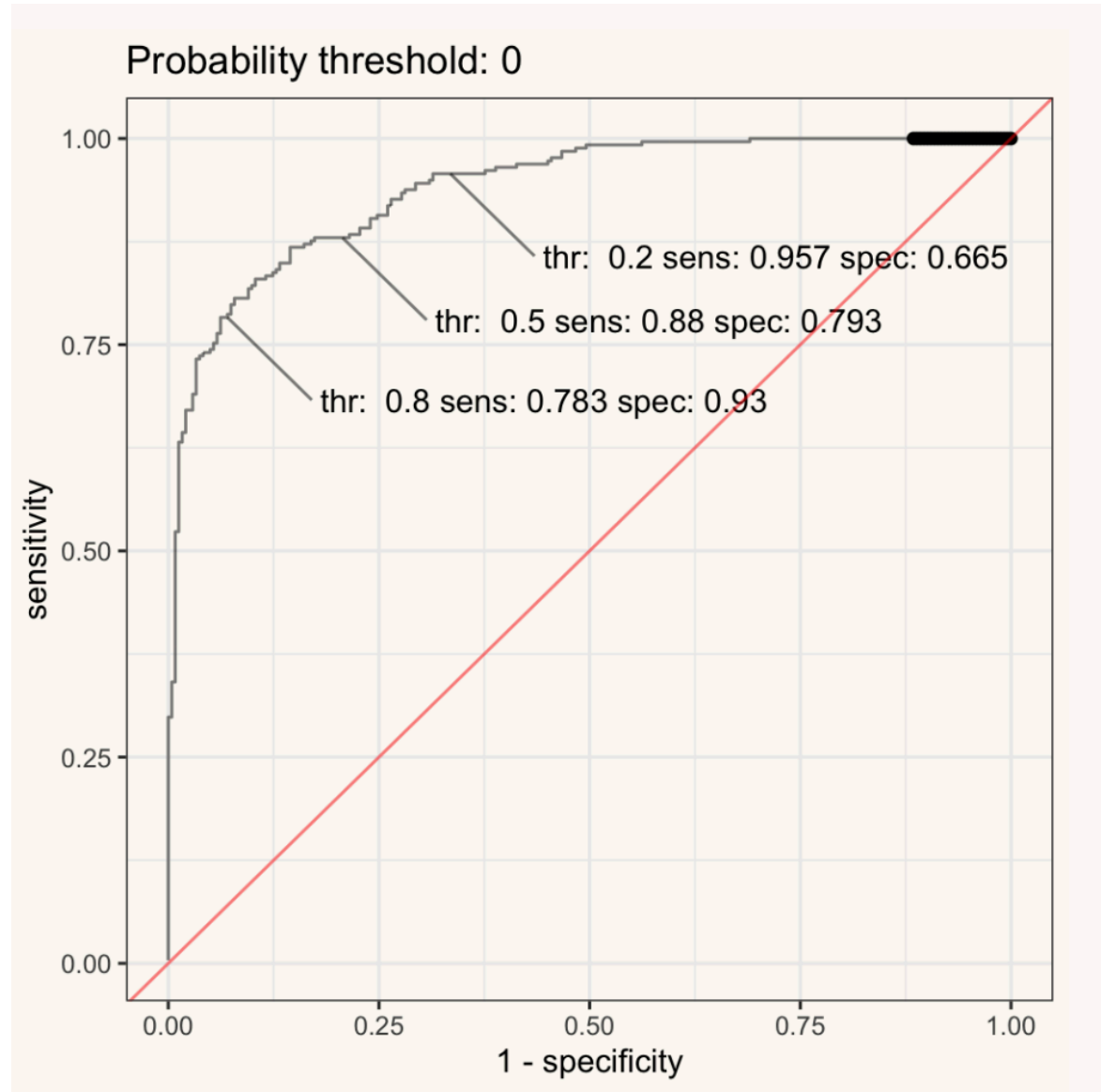
With two classes, the **Receiver Operating Characteristic (ROC) curve** can be used to estimate performance using a combination of sensitivity and specificity.

To create the curve, many alternative cutoffs are evaluated.

For each cutoff, we calculate the sensitivity and specificity.

The ROC curve plots the sensitivity (True Positive Rate) versus  $1 - \text{specificity}$  (False Positive Rate)

# The ROC Curve



# AUC

**Integral Relationship:** The AUC (Area Under the Curve) represents the integral or the total area under the ROC (Receiver Operating Characteristic) curve. It quantifies the overall ability of the model to discriminate between positive and negative classes across all possible thresholds, with a higher AUC indicating better model performance and discrimination capability.



# Classification Metrics with yardstick

```
1 data(two_class_example)
2 tibble(two_class_example)
```

```
#> # A tibble: 500 × 4
#>   truth    Class1    Class2 predicted
#>   <fct>    <dbl>    <dbl> <fct>
#> 1 Class2 0.00359 0.996   Class2
#> 2 Class1 0.679    0.321   Class1
#> 3 Class2 0.111    0.889   Class2
#> 4 Class1 0.735    0.265   Class1
#> 5 Class2 0.0162   0.984   Class2
#> 6 Class1 0.999    0.000725 Class1
#> 7 Class1 0.999    0.000799 Class1
#> 8 Class1 0.812    0.188   Class1
#> 9 Class2 0.457    0.543   Class2
#> 10 Class2 0.0976   0.902   Class2
#> # i 490 more rows
```

# Hard Prediction Metrics

```
1 # A confusion matrix:  
2 conf_mat(two_class_example,  
3         truth = truth,  
4         estimate = predicted)
```

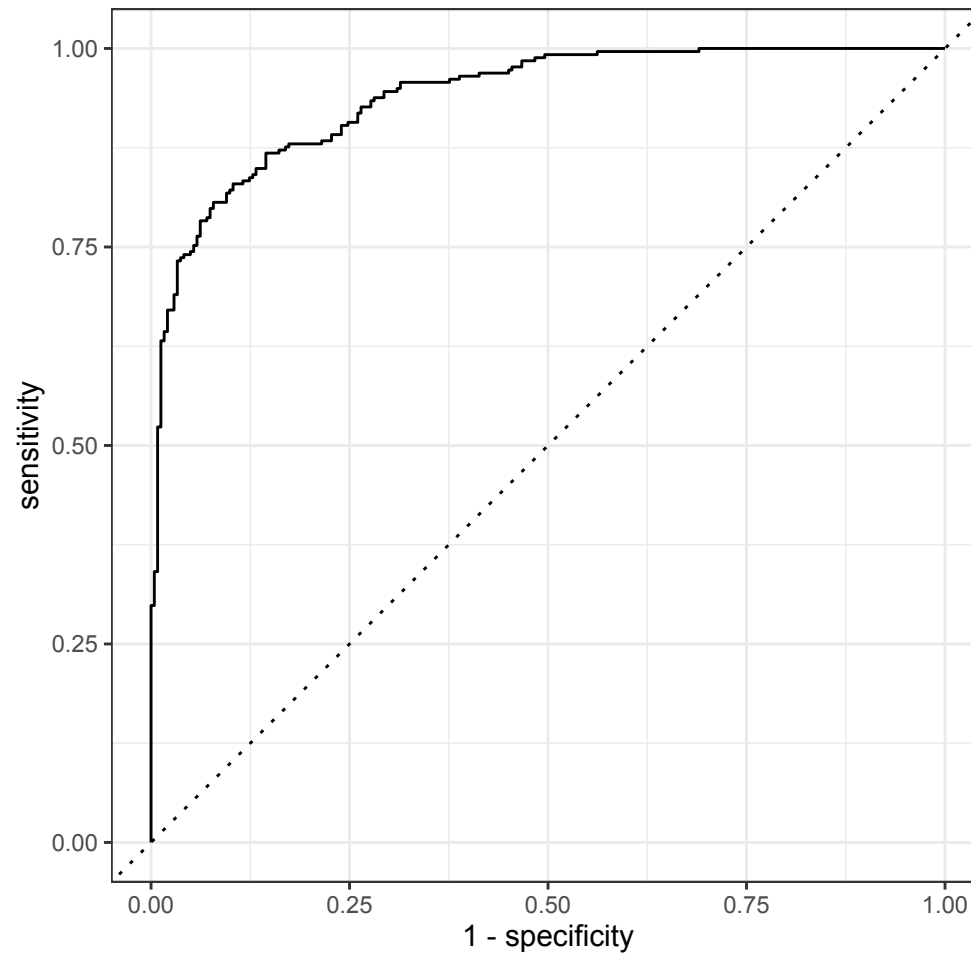
```
#>           Truth  
#> Prediction Class1 Class2  
#>      Class1    227    50  
#>      Class2     31   192
```

```
1 cm <- metric_set(accuracy,  
2                 sens,  
3                 spec)  
4 cm(two_class_example,  
5     truth = truth,  
6     estimate = predicted)
```

```
#> # A tibble: 3 × 3  
#>   .metric .estimator .estimate  
#>   <chr>    <chr>      <dbl>  
#> 1 accuracy binary     0.838  
#> 2 sens     binary     0.880  
#> 3 spec     binary     0.793
```

# ROC Curve

```
1 two_class_curve <- roc_curve(two_class_example, truth, Class1)
2 two_class_curve %>% autoplot()
```



# Calculating AUC

```
1 roc_auc(two_class_example, truth, Class1)
```

```
#> # A tibble: 1 × 3  
#>   .metric .estimator .estimate  
#>   <chr>   <chr>      <dbl>  
#> 1 roc_auc binary      0.939
```

# The Receiver Operating Characteristic (ROC) Curve

The ROC curve has some major advantages:

- It can allow models to be optimized for performance before a definitive cutoff is determined.
- It is *robust* to class imbalances; no matter the event rate, it does a good job at characterizing model performance.
- The ROC curve can be used to pick an optimal cutoff based on the trade-offs between the types of errors that can occur.

When there are two classes, it is advisable to focus on the area under the ROC curve instead of sensitivity and specificity.

Once an acceptable model is determined, a proper cutoff can be determined.

# Live Coding

# Classification toolkit (ISLR Ch. 4)

- **Logistic regression** (today's workhorse): flexible + interpretable
- **LDA / QDA**: model class-conditional distributions, then apply Bayes
- **KNN**: "predict like your neighbors" (non-parametric)
- **Naive Bayes**: simple generative model with strong independence assumption

# Two big families: discriminative vs generative

- **Discriminative:** model  $P(Y \mid X)$  directly
  - logistic regression
- **Generative:** model  $P(X \mid Y)$  and  $P(Y)$ , then use Bayes' rule
  - LDA, QDA, Naive Bayes



## Bayes theorem for classification

Thomas Bayes was a famous mathematician whose name represents a big subfield of statistical and probabilistic modeling. Here we focus on a simple result, known as Bayes theorem:

$$\Pr(Y = k|X = x) = \frac{\Pr(X = x|Y = k) \cdot \Pr(Y = k)}{\Pr(X = x)}$$

One writes this slightly differently for discriminant analysis:

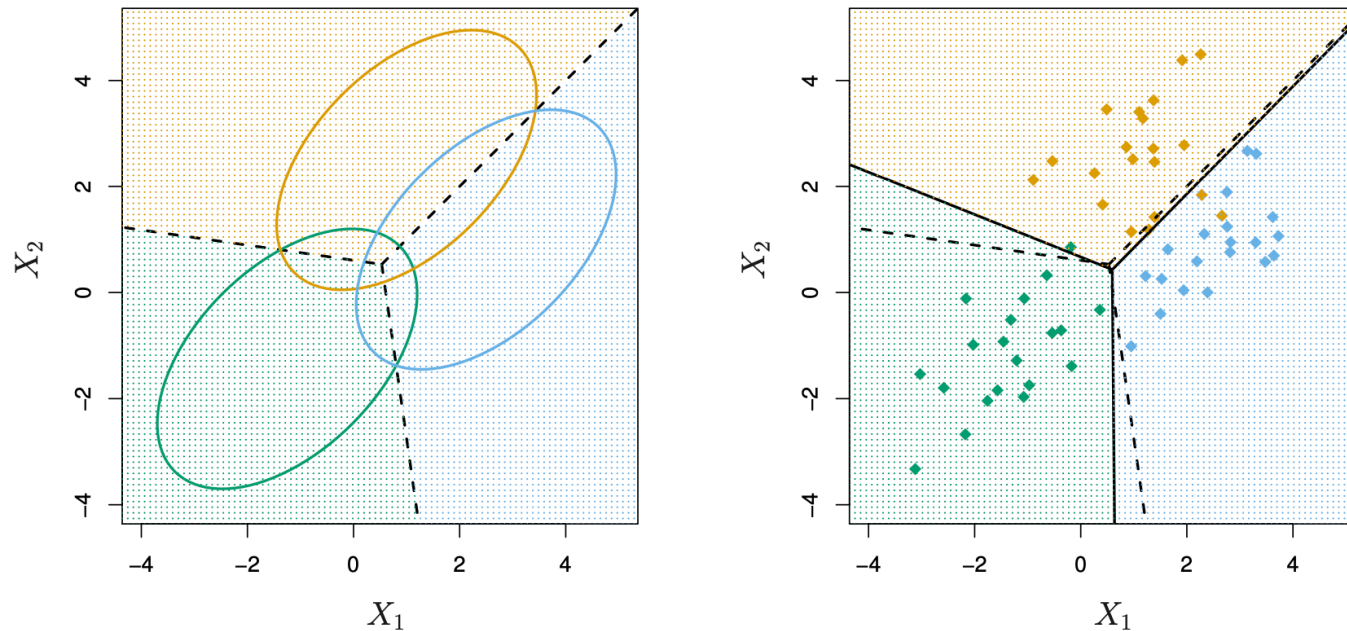
$$\Pr(Y = k|X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^K \pi_l f_l(x)}, \quad \text{where}$$

- $f_k(x) = \Pr(X = x|Y = k)$  is the *density* for  $X$  in class  $k$ . Here we will use normal densities for these, separately in each class.
- $\pi_k = \Pr(Y = k)$  is the marginal or *prior* probability for class  $k$ .

# LDA (Linear Discriminant Analysis)

- Assumes  $X \mid Y = k$  is multivariate normal
- Assumes **shared covariance** across classes
- Leads to a **linear** decision boundary

## Illustration: $p = 2$ and $K = 3$ classes



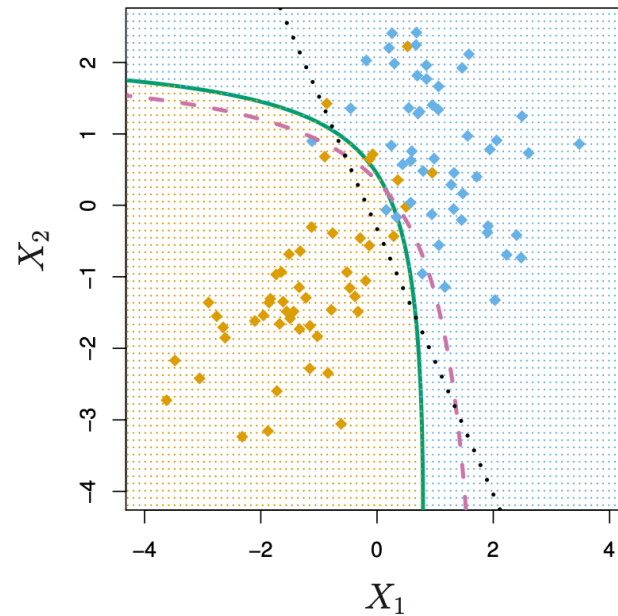
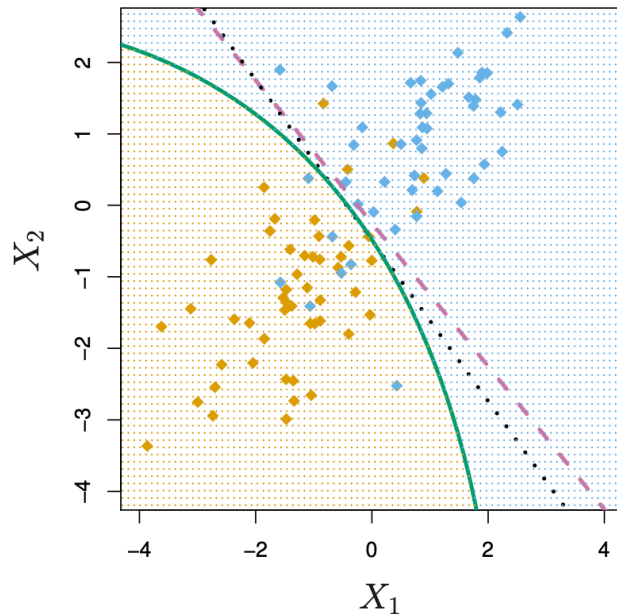
Here  $\pi_1 = \pi_2 = \pi_3 = 1/3$ .

The dashed lines are known as the *Bayes decision boundaries*.  
Were they known, they would yield the fewest misclassification errors, among all possible classifiers.

# QDA (Quadratic Discriminant Analysis)

- Same normality idea as LDA
- Allows **different covariance** per class
- More flexible → can overfit with limited data

## Quadratic Discriminant Analysis



$$\delta_k(x) = -\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1}(x - \mu_k) + \log \pi_k - \frac{1}{2} \log |\Sigma_k|$$

Because the  $\Sigma_k$  are different, the quadratic terms matter.

# KNN (k-nearest neighbors)

- No parametric form for  $P(Y \mid X)$
- Predict based on labels of the  $k$  nearest observations
- Key tuning knob:  $k$  (small  $k$  = flexible, large  $k$  = smoother)

# Naive Bayes

- Generative approach using Bayes' rule
- Assumes features are **conditionally independent** given the class
- Often works well in high dimensions (e.g., text), decent baseline elsewhere

# Summary: when might each show up?

- Logistic regression: default “first serious model”
- LDA/QDA: useful benchmarks; can be strong with small data / near-normal features
- KNN: intuitive, but sensitive to scaling and dimension
- Naive Bayes: fast baseline; works best when independence is “not too wrong”